

Linux Server Operations Lab

Phase 2 レビュー

Excelサーバーマスター - Excel × Python × Ansible × Linux

Phase 2 の目標：

人が保守しやすく、次のPhaseでPythonから読み取りやすい、信頼できるExcelサーバーマスターを作成する。

1. 作成したもの

ブック：Linux_Server_Operations_Lab.xlsx

シート：01_SERVER_MASTER

Excelテーブル名：tblServer

ServerID	Hostname	IP_Address	OS	Role	Environment
SV001	tokyo	192.168.56.11	Ubuntu 24.04	web	DEV
SV002	yokohama	192.168.56.12	Ubuntu 24.04	web	DEV
SV003	omiya	192.168.56.13	Ubuntu 24.04	web	DEV

2. なぜこのPhaseが必要だったのか

- ・Excelは、人が扱うマスターデータ層です。各サーバーを識別し、どのように扱うべきかを示す情報を保持します。
- ・Phase 3ではPythonがこのデータを読み込みます。そのため、見た目の装飾よりも、列名や値が一貫していることが重要です。
- ・Ansible側に同じホスト情報を何度もハードコードするのは避けます。長期的にはExcelを正本（source of truth）として、そこからAnsible用データを生成することを目指します。

3. 各列の意味

ServerID：各サーバーを管理するための安定した識別子。HostnameやIPアドレスが変わっても、内部IDは変えずに運用できます。

Hostname：tokyo、yokohama、omiyaなどのLinuxホスト名。

IP_Address：後でPythonがAnsible Inventoryを生成する際に使う管理用IPアドレス。

OS：OS情報。将来的にOSごとの処理分岐に利用できます。

Role：サーバーの用途。web / app / dbを用意し、役割によって点検内容やPlaybookを変えられるようにします。

Environment：環境区分。DEV / STG / PRODを用意し、例えばDEVでは変更を許可し、PRODでは制限するなど、安全な自動化ルールに利用できます。

4. Excelで実施した操作

1. ブックとシートを作成：Linux_Server_Operations_Lab.xlsxを作成し、最初のシート名を01_SERVER_MASTERへ変更。
2. ヘッダーを作成：1行目にServerID、Hostname、IP_Address、OS、Role、Environmentを入力。
3. 3台のサーバーを登録：tokyo、yokohama、omiyaと、管理用IP 192.168.56.11～13を登録。
4. Excelテーブル化：A1:F4をテーブルに変換し、データ範囲として扱いやすくした。
5. tblServerという名前を設定：Table1のような曖昧な名前を避け、用途が分かる名前にした。
6. Environmentに入力規則を設定：DEV / STG / PRODだけを選択できるようにした。
7. Roleに入力規則を設定：web / app / dbだけを選択できるようにした。

5. データ入力規則が重要な理由

自動化では文字列をそのまま値として扱います。たとえば、次の値は人には同じ意味に見えても、プログラムには別の値として扱われる可能性があります。

DEV
dev
Dev
DEVELOPMENT

ドロップダウンリストを使えば、Pythonがデータを受け取る前のExcel入力時点で表記ゆれを防げます。これは「入力層で先に検証する」という考え方の一例です。

6. 4つの技術の役割分担

レイヤー	このプロジェクトでの役割
Excel	人が管理するサーバーマスター。後のPhaseでは運用の操作盤にもなる。
Python	Excelデータの読込・検証、Ansible用データへの変換、Ansible実行、結果解析、Excelへの書き戻しを担当。
Ansible	Linuxサーバーへ接続し、複数ホストに対して点検や変更を統一的に実行。
Linux	実際の管理対象。サービス、ユーザー、ファイル、CPU、メモリ、ディスク、ログ、ネットワーク、パッケージなど。

7. Phase 2でLinux / Python / Ansibleが何をしたか

Linux：このPhaseではLinux操作は実施していません。既存の検証環境のホスト情報をExcelへ登録しました。

Python：まだ実行していません。Phase 2では、Pythonが安全に読める入力データを準備しました。

Ansible：まだ実行していません。後のPhaseで、このテーブルをInventory生成元として利用します。

Excel：このPhaseの主役です。構造化されたサーバーマスターとして機能するようになりました。

8. コマンド・Pythonコード・Ansible YAML・Excel関数

実行コマンド：なし。このPhaseはExcel内で完結しています。

Pythonコード：まだありません。次のPhaseからこのブックをPythonで読み始めます。

Ansible YAML：まだありません。Inventory生成は後のPhaseで行います。

Excel関数：このPhaseでは不要でした。重要だった機能は「テーブル」と「データの入力規則」です。関数は、運用上の明確な目的が出てきた段階で追加します。

9. 結果

サーバーマスターが、Pythonで読み込めるきれいな入力データとして準備できました。

ServerID	Hostname	IP_Address	OS	Role	Environment
SV001	tokyo	192.168.56.11	Ubuntu 24.04	web	DEV
SV002	yokohama	192.168.56.12	Ubuntu 24.04	web	DEV
SV003	omiya	192.168.56.13	Ubuntu 24.04	web	DEV

RoleとEnvironmentにはドロップダウンを設定し、よくある表記ゆれを防げるようにしました。

10. トラブルシューティングと確認ポイント

- ・値にばらつきがある場合は、まずデータの入力規則が対象行まで適用されているか確認する。
- ・Pythonが想定外の値を読み込んだ場合は、Excelのヘッダー名を正確に比較する。特にIP_AddressやEnvironmentの表記を統一する。
- ・後から新しい行を追加した場合は、tblServerの外側に追加されていないか確認する。
- ・自動化が誤った環境を対象にした場合は、PythonやAnsibleを疑う前に、まずExcelの元データを確認する。

11. 学んだこと

- ・ 良い自動化は、良い入力データから始まる。
- ・ Excelは単なる報告書ではなく、自動化システムにおける「人が操作する入力層」になれる。
- ・ 安定したID、一貫したヘッダー、制御された入力値によって、後続のPythonやAnsible処理のトラブルを減らせる。
- ・ RoleやEnvironmentは飾りではなく、後に「どの自動化を実行してよいか」を判断する条件として使える。

12. 次のPhase

Phase 3：PythonからExcelを読み込む。最初の目標は、Linux_Server_Operations_Lab.xlsxをPythonから開き、tblServerに登録した3行をプログラムのデータとして取得できることを確認することです。ここから最初の本格的な連携が始まります。

Excel → Python → （後で） Ansible → Linux

13. note記事候補

- ・ なぜAnsible学習でExcelを「正本」にしたのか
- ・ Pythonが安全に読めるサーバー管理台帳をExcelで作る
- ・ DEV / STG / PRODの入力規則 - 小さなExcel設定で自動化ミスを防ぐ
- ・ Excel × Python × Ansible - コードを書く前に入力層を設計する